DOCUMENTATION TECHNIQUE

Documentation Technique & Architecture du Pipeline

Table des Matières

CLI To-Do List Manager - Technical Specification & Architecture Document

1. Executive Summary & Architecture Overview

1.1 Executive Brief

A Python-based CLI To-Do List Manager utilizing a local JSON file for persistent storage. The system implements a service-oriented architecture separating CLI dispatch, business logic, and file I/O to manage task lifecycles including creation, completion, and removal.

1.2 Maturity Assessment

The project is structurally sound with a high health index and a clear execution roadmap. While formal acceptance criteria are missing for individual tasks, the presence of 'Independent Tests' for each user story provides sufficient validation gates. The project is READY for execution.

1.3 Technical Stack

- Python
- pyproject.toml

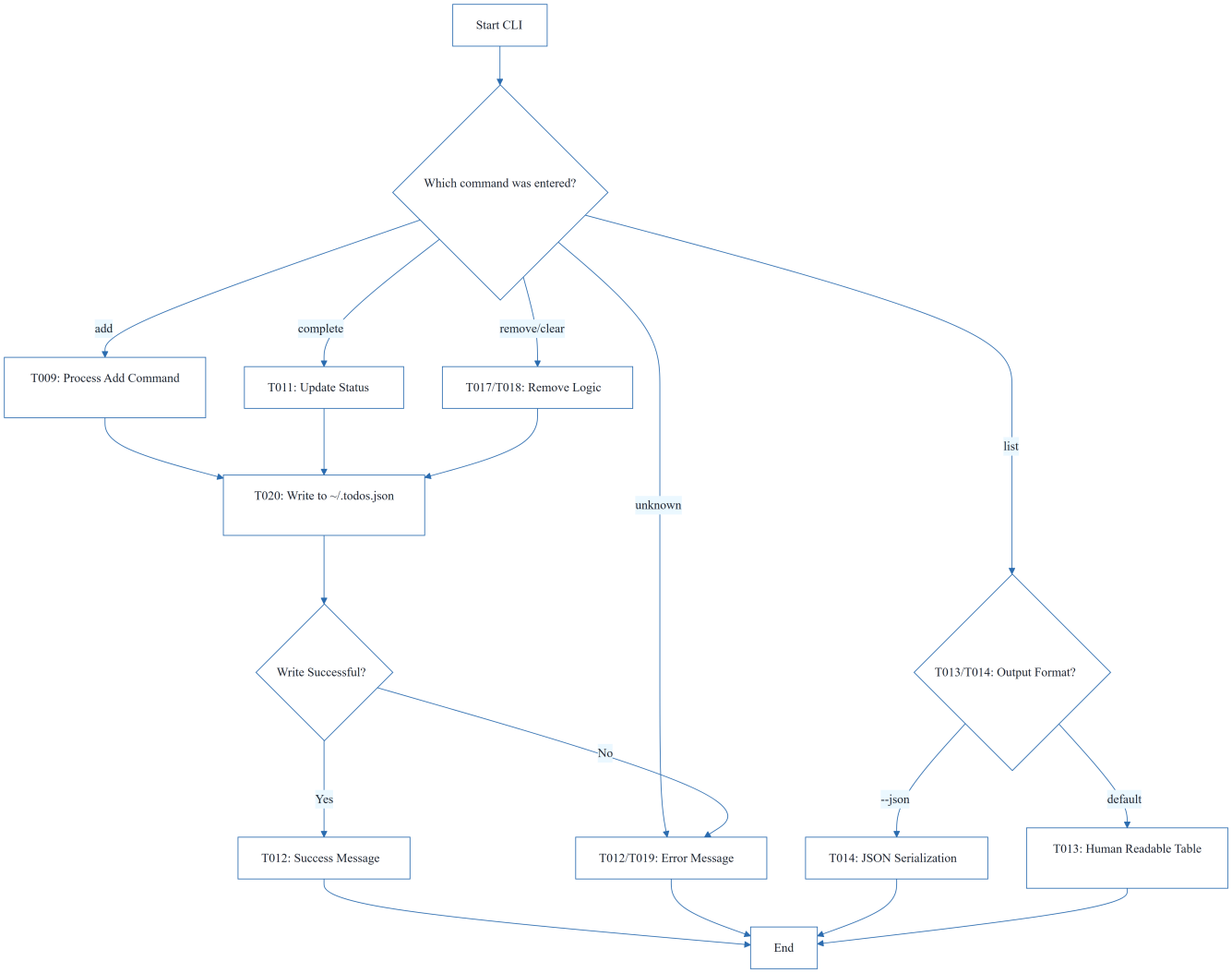
1.4 Architectural Constraints

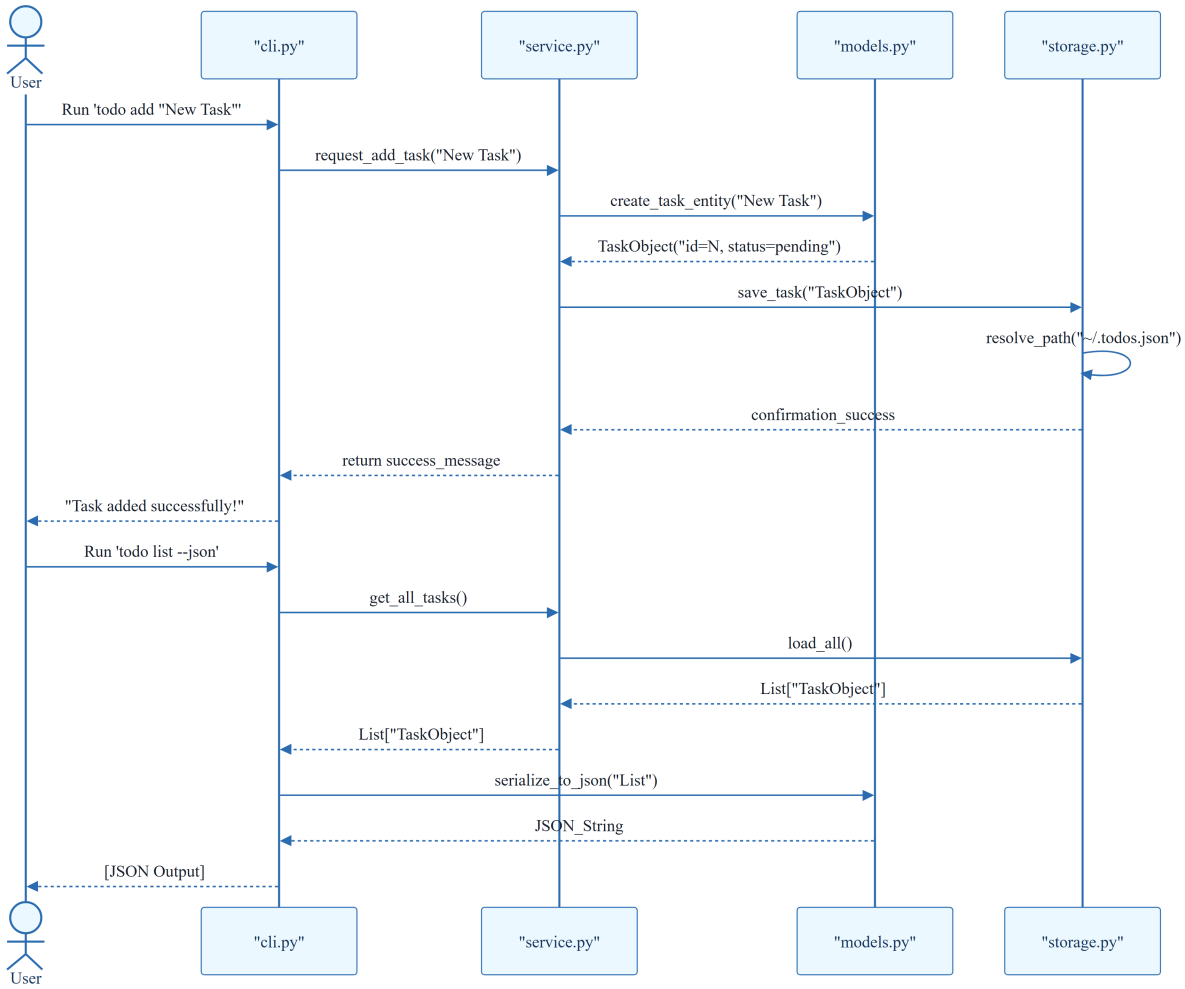
- Sequential ID assignment for task creation.
- Strict JSON serialization for data persistence.
- Storage path resolution targeting `~/.todos.json`.
- Human-readable and JSON output modes for task listing.
- Blocking dependency: Foundational phase must be complete before any User Story implementation.

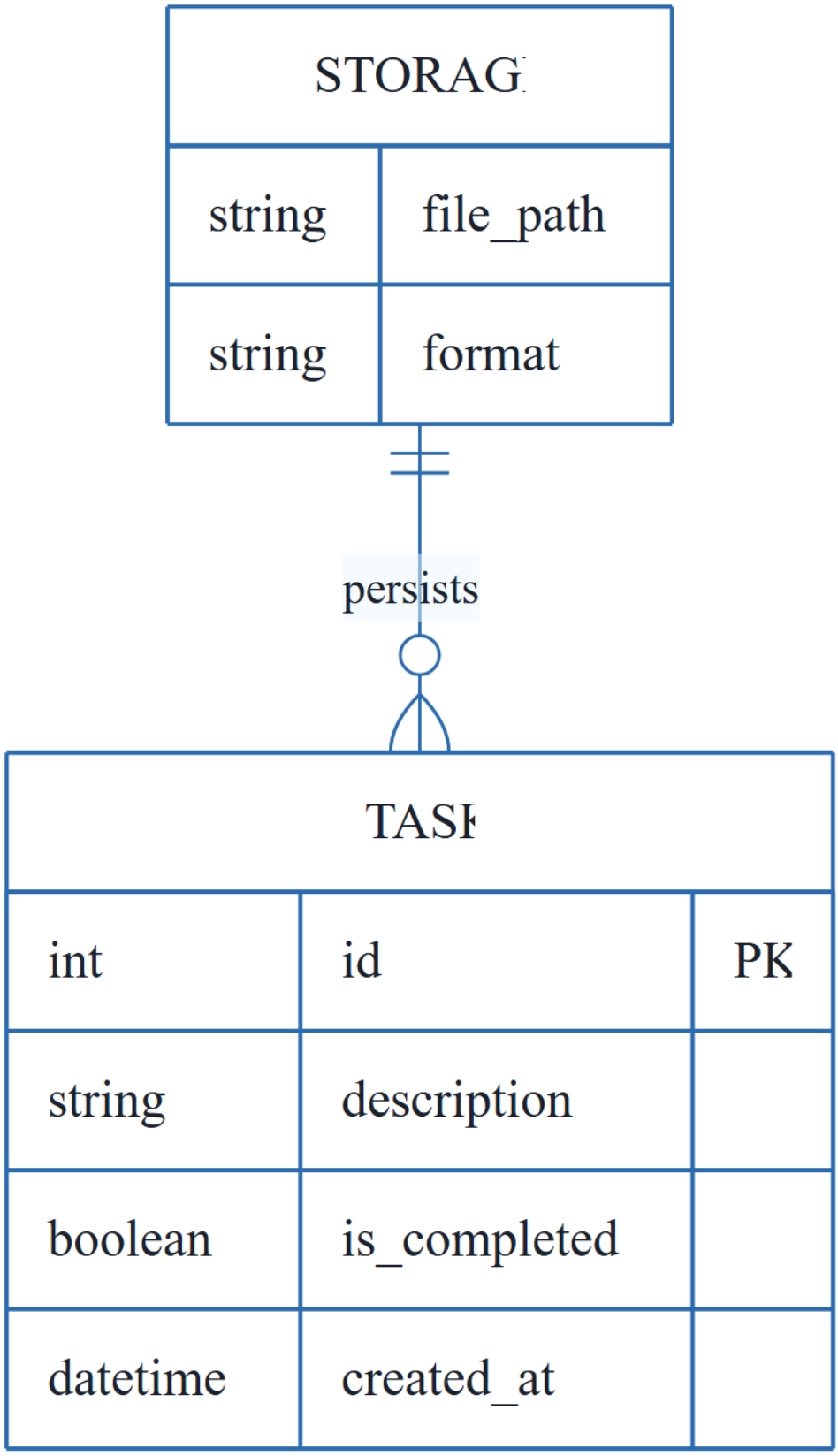
1.5 Critical Dependencies

- JSON file system I/O for `~/.todos.json`.
- Sequential ID generation logic in `models.py`.
- Referential integrity between CLI commands and service-layer utilities.
- `pyproject.toml` entry points for CLI execution.
- Cascading dependency: Phase 6 (Polish) requires completion of US1, US2, and US3.

2. Architecture Workflows

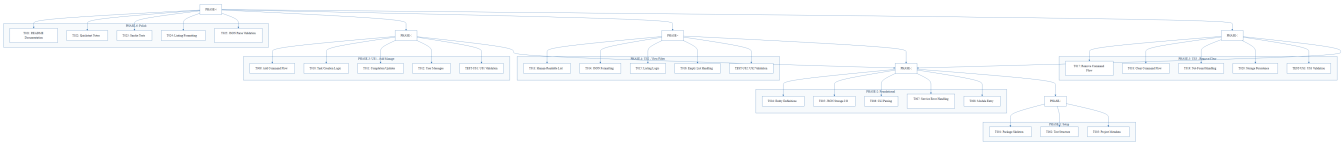

Start CLI
Which command was entered?
add
complete
remove/clear
unknown
list
T009: Process Add Command
T011: Update Status
T017/T018: Remove Logic
T020: Write to ~/.todos.json
Write Successful?
Yes
No
T012: Success Message
T012/T019: Error Message
T013/T014: Output Format?
--json
default
T014: JSON Serialization
T013: Human Readable Table
End


User
"cli.py"
"service.py"
"models.py"
"storage.py"
Run 'todo add "New Task"'
request_add_task("New Task")
create_task_entity("New Task")
TaskObject("id=N, status=pending")
save_task("TaskObject")
resolve_path("~/.todos.json")
confirmation_success
return success_message
"Task added successfully!"
Run 'todo list --json'
get_all_tasks()
load_all()
List["TaskObject"]
List["TaskObject"]
serialize_to_json("List")
JSON_String
[JSON Output]
User
"cli.py"
"service.py"
"models.py"
"storage.py"

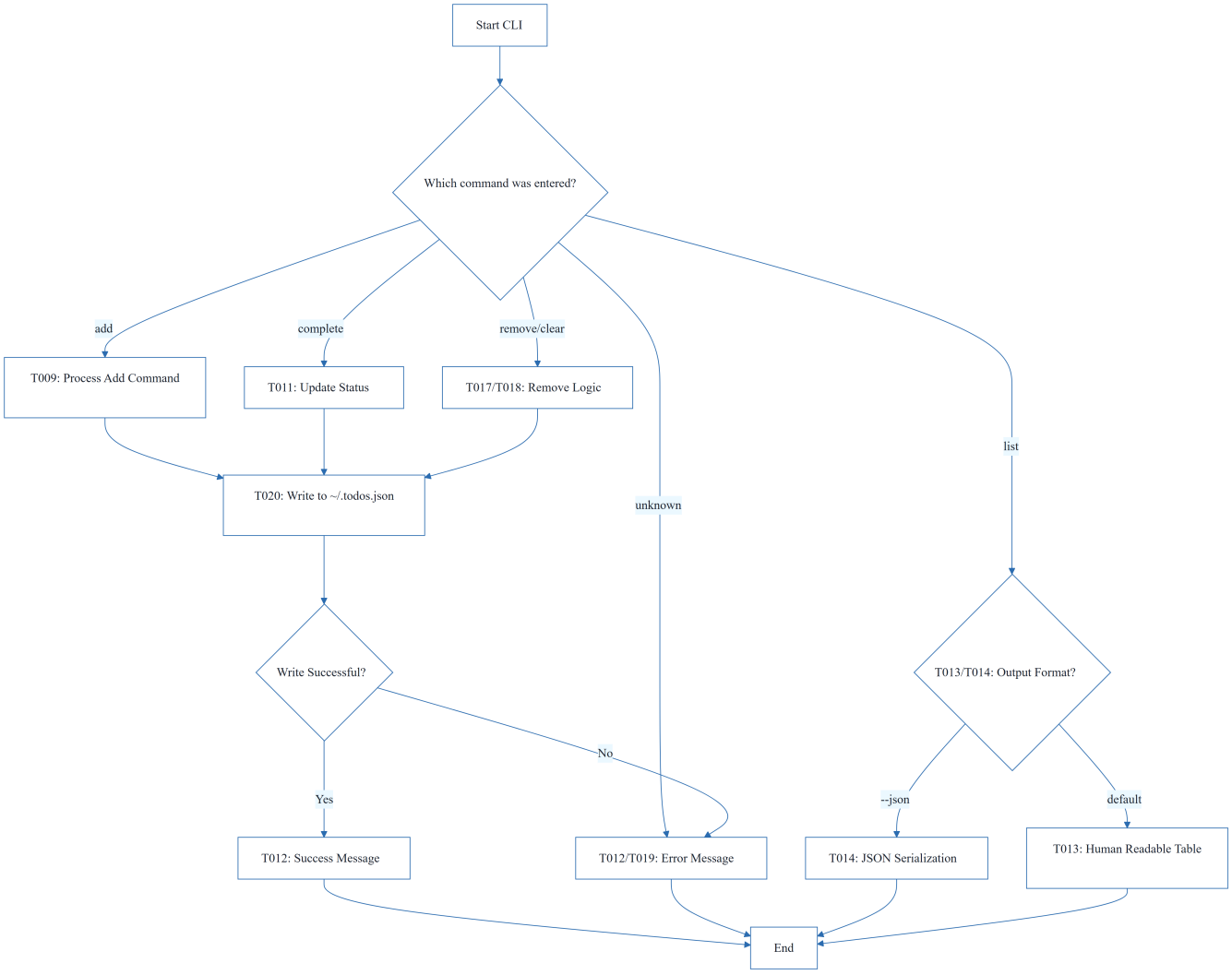


& Visual Diagrams

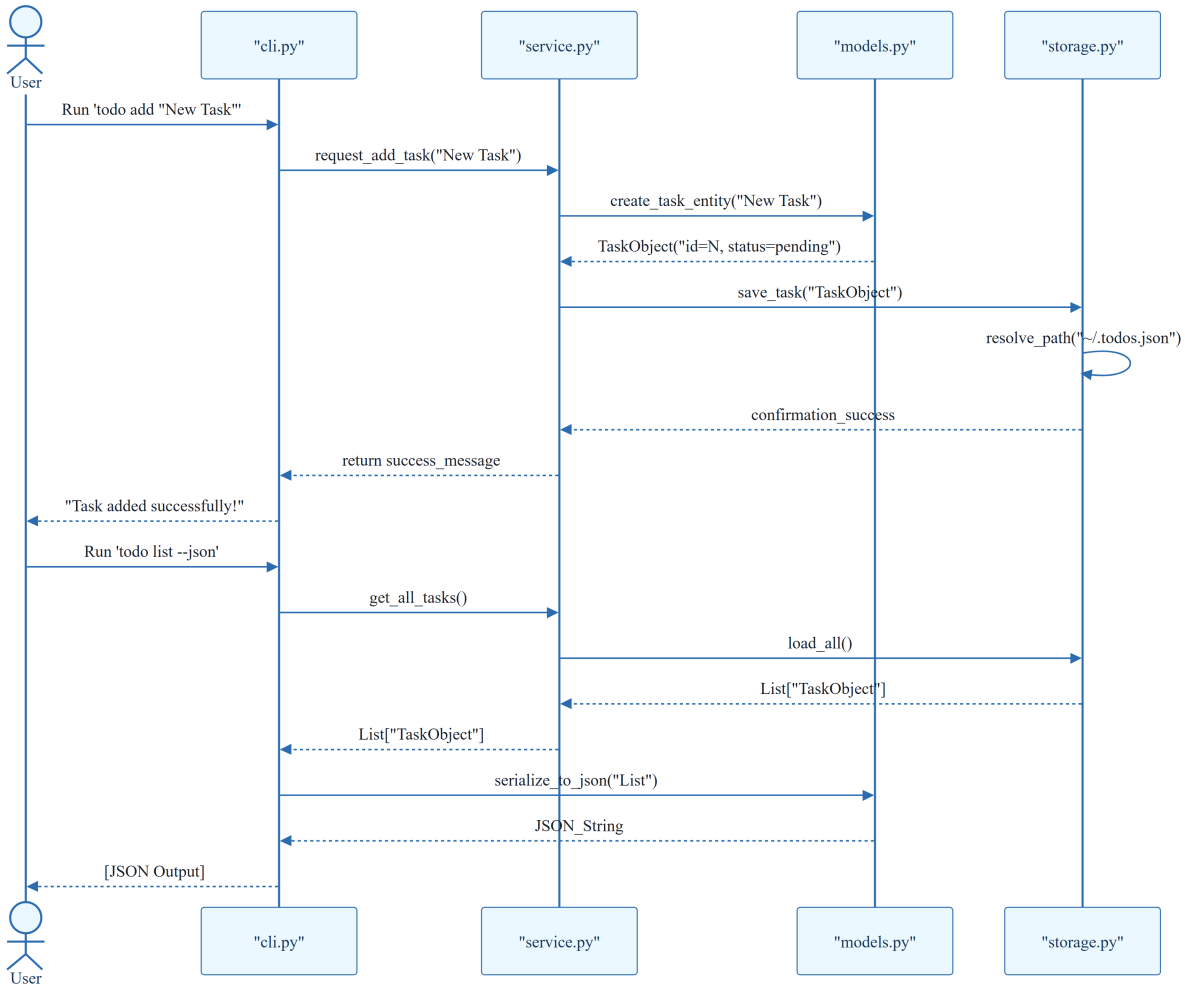
2.1 Project Implementation Roadmap & Traceability



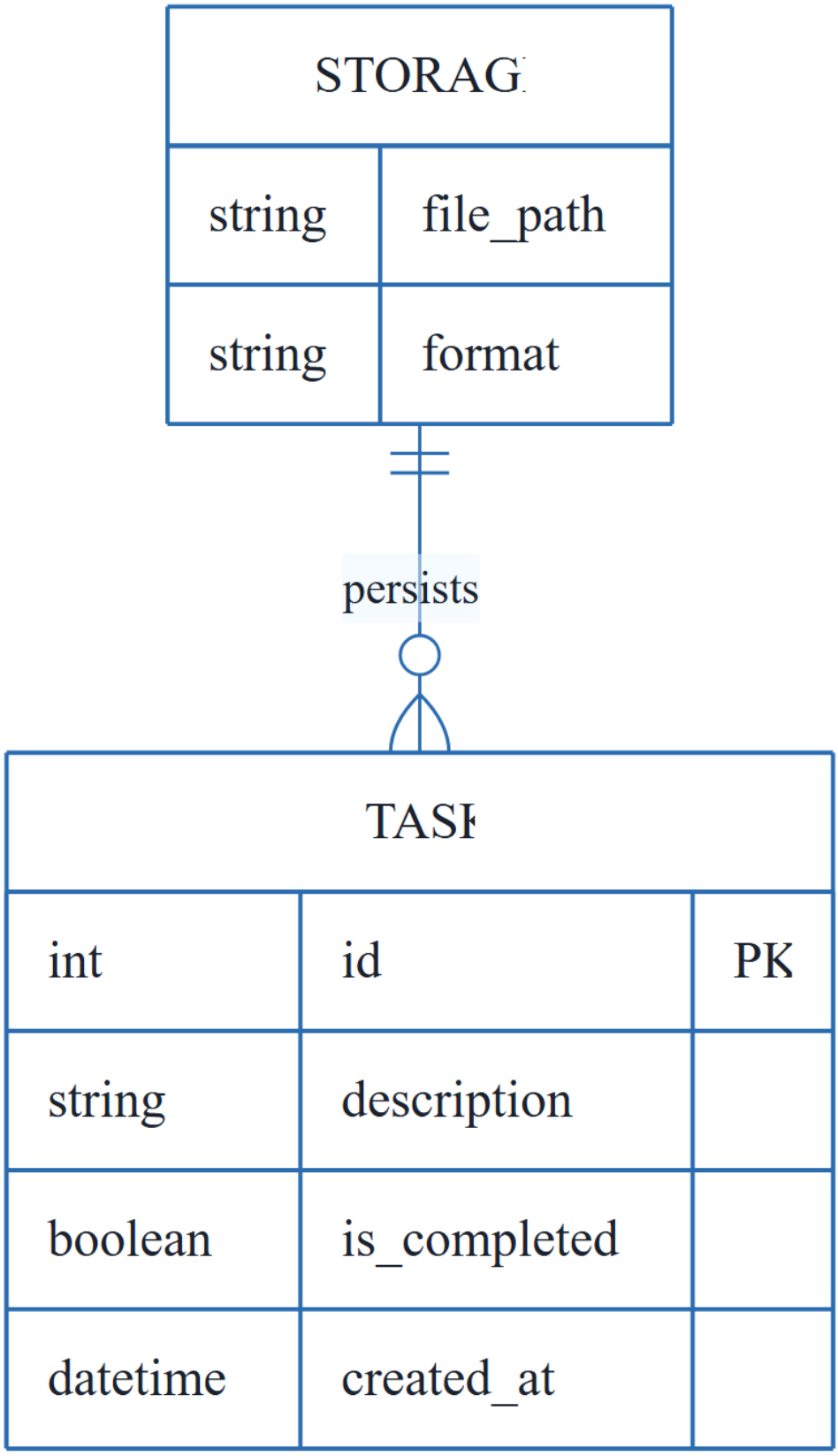
2.2 CLI Command Execution Workflow



2.3 Task Management Sequence



2.4 CLI To-Do Data Model



3. Detailed Technical Specifications & Business Rules

3.1 Requirements Traceability

ID	Description	Source Phase	Status
T001	Create the Python package skeleton in src/todo_manager/**init**.py, **main**.py, cli.py, models.py, storage.py, and service.py	PHASE-1	completed
T002	Create the test directory structure in tests/unit/ and tests/integration/	PHASE-1	completed
T003	Add project metadata and tooling entry points in pyproject.toml	PHASE-1	completed
T004	Implement task entity definitions and serialization helpers in src/todo_manager/models.py	PHASE-2	completed
T005	Implement JSON storage path resolution and file I/O helpers in src/todo_manager/storage.py	PHASE-2	completed
T006	Implement shared command-line parsing and top-level dispatch in src/todo_manager/cli.py	PHASE-2	completed
T007	Implement shared service-layer error handling and task collection utilities in src/todo_manager/service.py	PHASE-2	completed
T008	Define module entry behavior for python -m todo*manager in src/todo*manager/**main**.py	PHASE-2	completed
T009	Implement the `add` command flow in src/todo*manager/cli.py and src/todo*manager/service.py	PHASE-3	completed
T010	Implement task creation, sequential ID assignment, and `created_at` population in src/todo_manager/models.py	PHASE-3	completed

ID	Description	Source Phase	Status
T011	Implement completion updates for existing tasks in src/todo_manager/service.py	PHASE-3	completed
T012	Add user-facing success and error messages for add and complete operations in src/todo_manager/cli.py	PHASE-3	completed
T013	Implement human-readable task listing output in src/todo_manager/cli.py	PHASE-4	pending
T014	Implement `--json` output formatting in src/todo*manager/cli.py using the task serialization helpers in src/todo*manager/models.py	PHASE-4	completed
T015	Add listing logic that preserves task order and status fields in src/todo_manager/service.py	PHASE-4	completed
T016	Handle the empty-list case with a clear message in src/todo_manager/cli.py	PHASE-4	completed
T017	Implement the `remove` command flow in src/todo*manager/cli.py and src/todo*manager/service.py	PHASE-5	pending
T018	Implement the `clear` command flow for removing completed tasks in src/todo_manager/service.py	PHASE-5	pending
T019	Add not-found handling for invalid task IDs in src/todo_manager/cli.py	PHASE-5	pending
T020	Ensure storage writes persist removals and clears safely in src/todo_manager/storage.py	PHASE-5	pending
T021	Document the CLI usage and storage behavior in README.md	PHASE-6	pending
T022	Add quickstart verification notes and examples in specs/001-cli-todo-manager/quickstart.md	PHASE-6	pending

ID	Description	Source Phase	Status
T023	Run a manual smoke test of add, list, complete, remove, and clear against ~/.todos.json	PHASE-6	pending
T024	Verify linting and formatting expectations for the source files in src/todo_manager/	PHASE-6	pending
T025	Confirm the generated JSON output remains parseable for the todo list --json path	PHASE-6	pending
TEST-US1	Run `todo add "Task description"` and `todo complete` , then confirm the stored task list reflects the new task and its completion status.	PHASE-3	N/A
TEST-US2	Run `todo list` and `todo list --json` and confirm the output is readable or parseable JSON while showing the full task collection.	PHASE-4	N/A
TEST-US3	Run `todo remove` and `todo clear`, then confirm the targeted task or completed tasks are removed while other tasks remain intact.	PHASE-5	N/A

3.2 Security Rules

- No specific security constraints defined; however, input sanitization for CLI arguments is recommended to prevent unexpected behavior during JSON serialization.

3.3 Data Models

- **Task Entity**:
- `id` (Integer, PK): Sequential unique identifier.
- `description` (String): The task content.
- `is_completed` (Boolean): Completion status.
- `created_at` (DateTime): Timestamp of creation.
- **Storage**:
- `file_path` (String): Path to `~/.todos.json`.
- `format` (String): JSON.

4. Project Governance & Structural Gaps

4.1 Structural Gaps

Gap	Priority	Remediation Advice
Acceptance Criteria	MEDIUM	While 'Independent Tests' are provided, formal acceptance criteria for each task would improve validation.
Security & Performance Constraints	LOW	The project is a simple CLI tool, but constraints on file size or input sanitization could be added.
Open Questions & Uncertainties	LOW	No open questions were identified in the source document.

4.2 Remediation & Workflow

1. **Validation**: Implement formal acceptance criteria for each task in Phase 6.
2. **Hardening**: Add basic input validation in `cli.py` to ensure task descriptions are not empty.
3. **Verification**: Execute the smoke tests (T023) as the final gate before project closure.

5. Technical & Domain Glossary (Terminology Reference)

Term	Category	Context Anchor	Project Definition
Checkpoint	TECHNICAL_STACK	PHASE-2	A synchronization gate ensuring all blocking infrastructure is verified before parallel feature development commences.
Foundational	TECHNICAL_STACK	PHASE-2	The core shared infrastructure layer that provides essential serialization and I/O capabilities required by all subsequent features.
Goal	BUSINESS_DOMAIN	PHASE-3	The primary functional objective a user must achieve within a specific feature set.
ID	BUSINESS_DOMAIN	T010	A unique sequential numeric identifier assigned to each entry for precise targeting during updates or removals.

Term	Category	Context Anchor	Project Definition
JSON	TECHNICAL_STACK	T005	The lightweight data-interchange format used for persistent storage in the home directory and for machine-readable output.
MVP	BUSINESS_DOMAIN	PHASE-3	The minimum viable product consisting of the absolute essential capabilities to create and mark entries as finished.
Organization	TECHNICAL_STACK	Tasks: CLI To-Do List Manager	The structural grouping of implementation units by user story to ensure independent testability.
Prerequisites	TECHNICAL_STACK	Tasks: CLI To-Do List Manager	The set of mandatory design and contract documents required before implementation begins.
README	TECHNICAL_STACK	T021	The primary documentation file detailing usage instructions and storage behavior.
Setup	TECHNICAL_STACK	PHASE-1	The initial phase involving package skeleton creation and tooling configuration.
T016	TECHNICAL_STACK	T016	The specific implementation unit responsible for providing user feedback when no entries are available to display.
Tests	TECHNICAL_STACK	T002	The verification suite divided into unit and integration directories to ensure logic correctness.
population in	TECHNICAL_STACK	T010	The process of automatically assigning a timestamp to the creation field during entry instantiation.
todo clear	BUSINESS_DOMAIN	T018	The operation that purges all entries marked as finished from the persistent store.
todo list	BUSINESS_DOMAIN	T013	The operation that retrieves and displays all stored entries in either human-readable or machine-parseable formats.

Term	Category	Context Anchor	Project Definition
■■ CRITICAL	TECHNICAL_STACK	PHASE-2	A high-priority constraint indicating that subsequent work is strictly blocked until the current phase is fully validated.